

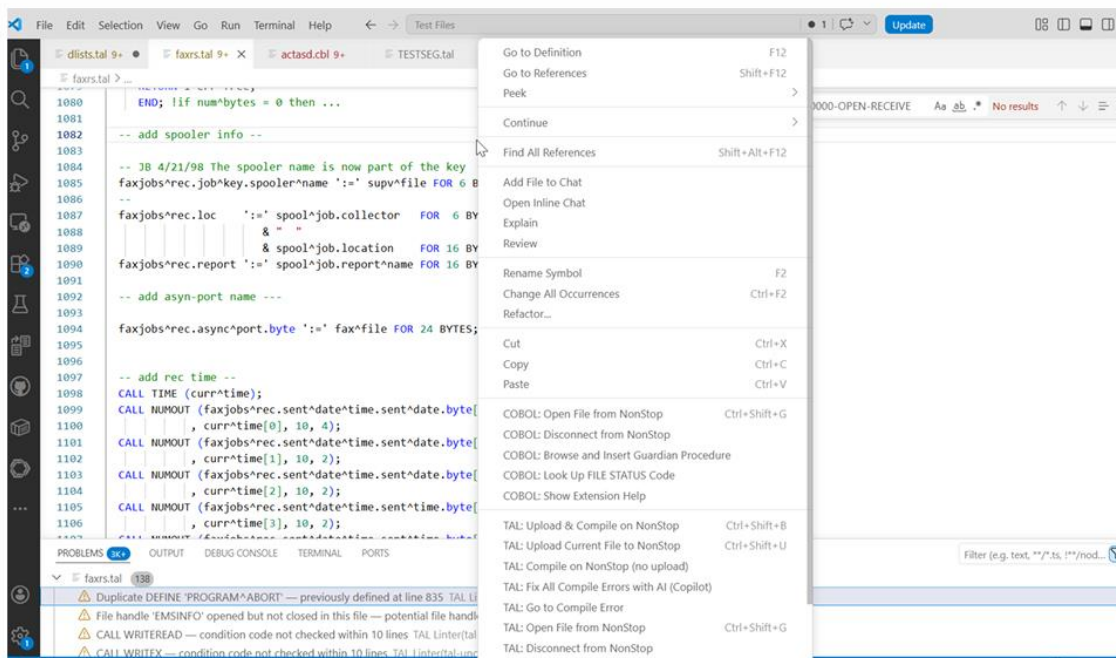


TAL Language Support

Architecture Overview

A deep dive into how VS Code, MCP Server, and NonStop work together to bring modern developer tooling to TAL and Guardian systems.

VS Code Integration – Nonstop TAL and COBOL



System Overview

Three tightly integrated layers form the backbone of the TAL development environment.

VS Code	MCP Server	NonStop
- TAL Extension - Syntax & IntelliSense - Real-time Linting - Claude Code Chat	- Tool Execution - Compilation - Guardian APIs - Database	- File System - TAL Compiler - Guardian Runtime - Execution

How It Works

 Code Editing - Real-time syntax highlighting - Code completion - Inline error detection - Navigate definitions	 Compilation - Upload TAL file to NonStop - Execute remote compiler - Capture results - Map errors to editor	 Chat Help - Ask questions in plain language - Query Guardian APIs - Generate code snippets - Explain errors
--	---	---

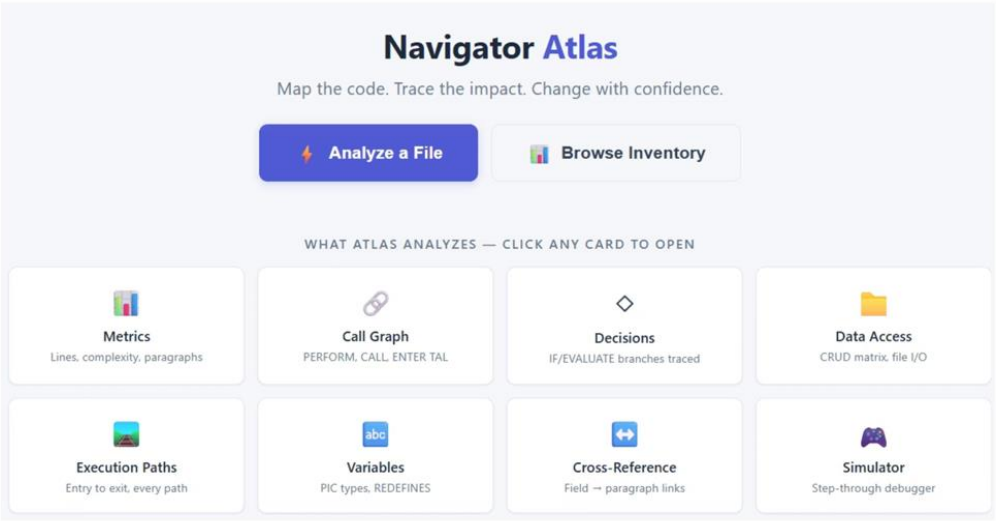


Navigator Atlas

Deterministic Static Analysis for HPE NonStop

See your COBOL, TAL, SCOBOL & TACL like never before — without guesswork, without AI hallucination, without sending a single line of code off-site.

- STATIC ANALYSIS
- NONSTOP NATIVE
- 100% DETERMINISTIC



Single-Program Report: Overview

Every program analyzed by Atlas gets a dedicated Report page — a single-screen summary that answers the most common questions without any digging.

Language & Lines

Auto-detected language with manual override badge. Exact line count including copybook expansion.

Paragraphs & Complexity

Total paragraph count and cyclomatic complexity score — your first signal of maintenance risk.

Issue Count

Dead code, unreachable paragraphs, and structural anomalies surfaced automatically.

Report Tabs

- Overview — metadata & issue summary
- Source — navigable source viewer
- Calls — paragraph call tree
- Data — file & field access
- Diagrams — visual graphs
- Variables — data item index
- Simulator — trace execution paths

Diagrams That Actually Help

Atlas generates eight distinct diagram types, all interactive in the browser and exportable as SVG for documentation or architecture reviews.



Call Graph

Paragraph-to-paragraph execution edges — see exactly who calls what.



Data Graph

Paragraphs ↔ files they read or write — map data flow at a glance.



Paragraph Flow

Control flow at the paragraph level — visualize branching and looping.



Redefines

COBOL data structure overlays — understand REDEFINES chains without guessing.



Flow / Sequence

Execution sequence and inter-program message flows rendered clearly.



Copy Tree & Heatmap

Copybook inclusion hierarchy and complexity heatmap for risk-at-a-glance.

Cross-Reference Matrix

The CrossRef Matrix gives you a single grid showing every program's relationship with every file in the folder — the kind of view that used to take weeks of manual documentation to produce.

Color-Coded Access Types

- **Green** — Read-only access
- **Red** — Write-only access
- **Yellow** — Read + Update (most common risk)
- **Full CRUD** — Complete access flagged for review

Interactive Drill-Down

Click any cell in the matrix to navigate directly to that program's detailed file access report. Filter the grid by program name, file name, or shared-file-only to focus on what matters for your current task.

The matrix makes shared-file risk visible at a glance — a critical input for any refactoring or migration plan.

Cross-Program Call Graph

The folder-level Call Graph lifts the view from individual programs to the entire subsystem — showing how programs call each other, where clusters form, and where orphans hide.

Subsystem Coloring

Nodes are colored by subsystem or category, making architectural boundaries immediately visible — even in a folder of 500 programs.

Call Type Edges

Edges are typed by call mechanism: CALL, ENTER TAL, SEND, and other inter-program communication patterns — each rendered distinctly.

Click-Through Drill-Down

Click any node in the graph to navigate directly to that program's individual report — seamlessly bridging the folder and program views.

Orphans & Tight Clusters

Unreachable programs (orphans) and tightly-coupled clusters surface immediately — both are key risk signals for modernization planning.